

Chapter 4 — State & Hooks

Chapter 4 — State & Hooks

State is what makes a static page interactive. This chapter introduces `useState`, `useMemo`, the orchestrator pattern, and the most-asked React interview concept of all: lifting state up.

What you'll learn

- What **state** is and why it's different from props.
- How `useState` works — the array-destructuring syntax, the setter, the re-render cycle.
- **Lifting state up** — the canonical React pattern for “two components share data.”
- The **orchestrator** component — where state lives in real apps.
- **`useMemo`** for derived data — when it's necessary, when it's not.
- **Cross-component navigation** — clicking in one tab moves to another.
- Why **state is a tree, not a graph** — and why that constraint is a feature, not a limitation.

Lessons in this chapter

1. [01-useState-mental-model.md](#) — Your first hook. What state is, what `useState` returns, the re-render cycle, the rules of hooks. The mental model that takes most beginners weeks to internalize.
2. [02-the-orchestrator-pattern.md](#) — Lifting state into the parent. Tab switching, cross-tab interactions, callbacks-down. This is the chapter where the app starts feeling like an app.
3. [03-useMemo-and-derived-data.md](#) — When (and when not) to memoize. Reference equality, identity stability, and when not optimizing is the right call.

Files you'll create or update

```
components/
├── SnookerFantasyLeague.tsx    ← UPDATED: orchestrator with state, tab nav, he
├── tabs/
│   ├── StandingsTab.tsx      ← UPDATED: receives teams via props, no longer sel
contained
│   ├── MatchesTab.tsx        ← NEW: stub for now
│   ├── PredictionsTab.tsx    ← NEW: stub for now
│   ├── PlayersTab.tsx        ← NEW: stub for now
│   └── AnalyticsTab.tsx      ← NEW: stub for now
```

The non-Standings tabs are stubs in this chapter (“Coming soon” placeholders). Chapters 5 and 6 fill them in.

New React/Next.js concepts introduced

- **useState** — the most-used hook in React.
- **The re-render cycle** — what triggers a component to re-execute.
- **Lifting state up** — moving state to a common ancestor.
- **Controlled components** — parent owns the data; child receives + dispatches updates.
- **useMemo** — caching expensive computations across renders.
- **Reference equality** vs **value equality**.
- **Rules of hooks** — and why they exist.

How long it should take

- 30–45 min reading per lesson
- 1–2 hours typing the code
- ~4 hours total

Why this chapter is the most important in the course

Honestly? Because **state is what gets you hired**.

Once you can answer “where does this state live?” correctly, every other React question gets easier. UI structure, data flow, performance, testing — they all derive from where the

state is. Junior interviews ask *“what’s a hook?”* — easy. Senior interviews ask *“why does the state live here and not there?”* — that’s this chapter.

If you only deeply read three chapters of this course, make it 1, 4, and 7. The rest is execution; these are the architecture.

Before you start

You should have:

- Chapter 3 complete: a fully working standings tab with stat cards, medal-color rank circles, per-round columns, Final Pick chip, FormBadge.
- `components/SnookerFantasyLeague.tsx` as a stub with `'use client'` and `<StandingsTab />` inside.
- `npm run dev` running and the page rendering correctly.

Open [01-useState-mental-model.md](#) when ready.